

Code Review Sênior — LLM Guardrails Proxy

Escopo: Repositório completo (/home/germano/Projects/LLM-Guardrails-Tutorial) **Contexto:** MVP de 2 dias para entrevista técnica — proxy LangGraph + FastAPI com validators (toxic, PII PT-BR, jailbreak em camadas, compliance LLM judge), RAG (Qdrant + sentence-transformers), Streamlit UI, structlog JSON. **Data:** 2026-05-26 **Recomendação:** APROVADO COM RESSALVAS

Sumário Executivo

O projeto demonstra rigor de design e atenção à segurança raros em projetos de guardrails LLM construídos em prazo apertado. O fail-closed é a prioridade certa na maior parte do pipeline (compliance judge, vector store, LLM provider degradam graciosamente), a arquitetura LangGraph é limpa, e a documentação é honesta sobre limitações em LIMITATIONS.md e nos 6 ADRs. As escolhas de trade-off (regex vs Presidio, e5 local vs Voyage, JSON logs vs Langfuse) estão bem justificadas. **Há, porém, três gaps críticos de fail-closed em validators ML que precisam ser corrigidos antes de produção.**

Pontos Positivos

Arquitetura e Validators

- **Fail-closed robusto no ComplianceValidator** (guardrails/validators/compliance.py:119-133): exceção da API Anthropic vira ValidatorResult(passed=False, score=1.0, error=ExceptionType) . Coberto por test_fail_closed_on_api_exception .
- **Degradação elegante do RAG** (guardrails/pipeline/nodes.py:96-106): falha de embedding/vector store retorna query_vec=None em vez de propagar.
- **Adapters LLM fail-closed** (guardrails/adapters/llm.py:84-85, 115-118): complete() retorna "" , complete_with_tools() retorna None em exception.
- **request_id via contextvars** (guardrails/api/app.py:94-102): bound na entrada, cleared no finally — sem vazamento entre requests.
- **Layered defense bem orquestrada** (guardrails/validators/jailbreak.py:91-158): substring fast-path (<5ms) antes do DeBERTa (<300ms); 21 keywords PT-BR/EN curadas.
- **StateGraph íntegro** (guardrails/pipeline/graph.py): 5 nodes, rotas condicionais explícitas após input e output guards, sem nodes órfãos.
- **Protocol-based adapters** (guardrails/adapters/{llm,embedding,vector_store}.py): @runtime_checkable torna a narrativa "swap para Bedrock" crível.
- **Cache control em prompts** (compliance.py:90 , llm.py:79): cache_control ephemeral pronto para prompt caching.

Observabilidade e Configuração

- **Structlog com sanitização** (observability/logger.py): PII redigido antes de logar, hash do input para forensics, latência por validator.
- **Config com expansão de env** (config.py:14): \${ANTHROPIC_API_KEY} sem hardcoded.

Testes e Qualidade

- **76 testes passando + 3 xfail declarados** alinhados com `LIMITATIONS.md` (CPF unformatted, telefones 11/9 dígitos).
- **Fixtures adversariais externas:** `jailbreak_external.jsonl` (JailbreakBench, MIT), `toxic_external` (HateBR, RealToxicityPrompts) — não-tautológicas para esses dois domínios.
- **Lint e formato 100% limpos:** `ruff check` e `ruff format --check` sem warnings em 66 arquivos.

Documentação e Processo

- **LIMITATIONS.md magistral:** gaps confirmados (não hipotéticos) por validator, com `"closed_loop": true` marcado nos JSONLs de PII/Compliance.
- **6 ADRs** cobrindo cada decisão pivô (abandono de guardrails-ai, Presidio, Voyage, Langfuse) com trade-off explícito.
- **CLAUDE.md como tabela viva** de decisões com data — rastreabilidade excelente.

Problemas Negativos

● Críticos

C1. ToxicValidator sem exception handling `guardrails/validators/toxic.py:38-55` — `self._model.predict(text)` pode lançar (OOM, device mismatch) e crasha o request com HTTP 500. Não há try/except. - **Impacto:** Fail-OPEN em sentido funcional (request morre). Input guard não pode quebrar. - **Correção:** try/except retornando `ValidatorResult(passed=False, score=1.0, details={"error": type(e).__name__})`, espelhando `ComplianceValidator`.

C2. JailbreakValidator sem exception no pipeline DeBERTa `guardrails/validators/jailbreak.py:126-128` — `self._pipeline(text)[0]` sem proteção. Modelo de ~500MB pode falhar a carregar/inferir. - **Impacto:** Em produção (cache miss, CUDA mismatch) o request morre. - **Correção:** try/except retornando fail-closed verdict.

C3. Compliance Judge parse de tool_use quebra silenciosamente `guardrails/validators/compliance.py:99` — `next(b for b in response.content if b.type == "tool_use")` levanta `StopIteration` se o modelo não usar a tool (ex: `stop_reason="end_turn"`). Isso acontece **antes** do `except` da linha 119, então não é capturado pelo fail-closed handler. - **Impacto:** Quebra fail-closed do validator mais crítico (compliance bancário). - **Correção:** `next(..., None)` + check explícito → fallback fail-closed se ausente.

● Médios

M1. HTTP 200 para blocks não documentado em OpenAPI `guardrails/api/app.py:104-140` — `CLAUDE.md` declara "block = HTTP 200 (policy decision)", mas o schema OpenAPI não evidencia isso. Cliente integrador pode interpretar 200 como sucesso. - **Correção:** ou documentar em `response_model /docstring`, ou usar HTTP 403 (semanticamente melhor).

M2. Lifespan não valida carregamento dos modelos ML `guardrails/api/app.py:76-87` — Se Detoxify/DeBERTa/embedding falham a carregar, `/health` ainda retorna 200, mas `/chat` morre. - **Correção:** smoke test (1 char) por validator no lifespan; falhar startup se algum modelo não carrega.

M3. PII regex sem checksum (CPF Módulo 11, Luhn em cartão) `guardrails/_pii_patterns.py:15-16` — aceita `000.000.000-00`. Declarado em `LIMITATIONS.md` e marcado como Extra, mas em contexto bancário é um gap relevante de discutir.

M4. DeBERTa English-dominant, PT-BR sub-medido `LIMITATIONS.md:79-80` — modelo `protectai/deberta-v3-base-prompt-injection-v2` treinado em EN. Tabela em `LIMITATIONS.md` mostra 12/12 PT-BR mas o N (22 samples totais) é estatisticamente fraco. - **Aceitável para MVP, mas dimensionar honestamente em narrativa.**

M5. Fixtures de Compliance e PII com closed-loop documentado `tests/adversarial/fixtures/compliance_handcrafted.jsonl` — rubrica + fixtures + judge do mesmo autor. Honestamente declarado em `LIMITATIONS.md`. Mitigação real exigiria Cohen's kappa com annotator independente (Extras).

M6. Qdrant depends_on: service_started (não service_healthy) `docker-compose.yml:15-17` — API pode subir antes do Qdrant estar pronto. Demo pode falhar silenciosamente. - **Correção:** adicionar `healthcheck` no Qdrant + `service_healthy`.

M7. Compliance Judge sensível a reformulação sem histórico `compliance.py:63` — Judge vê apenas o último output, sem histórico de turn. Beat 4 da demo medido em 1 único fraseamento.

 Leves

L1. retrieve node — `try/except` cobre embedding mas não `vector_store.search` (`nodes.py:96-106`).

L2. block_log node não persiste em DB — apenas structlog. Aceitável para MVP, declarado.

L3. Timeout do Compliance é fixo em 5s (`compliance.py:51`) — não expõe em config.

L4. batch_size=32 hardcoded no embedding (`adapters/embedding.py:32`).

L5. Prefixos query: / passage: do E5 documentados no docstring mas não no Protocol — implementador alternativo pode esquecer.

L6. Warning de DeprecationWarning do torch (JIT script) no pytest — ruído cosmético.

Validação Automatizada

Check	Resultado
<code>ruff check .</code>	✅ All checks passed
<code>ruff format --check .</code>	✅ 66 files already formatted
<code>pytest -m "not slow and not network and not adversarial"</code>	✅ 76 passed, 3 xfail (esperados)

Análise sob a lente "Building Rigorously"

- **§1 (closed validation loop)**: parcialmente mitigado. Jailbreak e Toxic usam fontes externas. Compliance e PII são hand-crafted — risco **declarado** em `LIMITATIONS.md` (boa prática §7).
 - **§3 (100% green é warning)**: 76/76 passando, mas a maior parte são unit tests com mocks. Adversarial smoke é pequeno (~22 samples jailbreak PT-BR). Mais red-teaming PT-BR daria mais confiança.
 - **§4 (doc drift)**: CLAUDE.md, ADRs e LIMITATIONS.md estão **consistentes com o código** observado. Sem drift visível.
 - **§6 (substring matching não é guardrail)**: respeitado — substring é fast-path explícito, layered com DeBERTa.
 - **§7 (limitações honestas)**: exemplar.
 - **§8 (volume de artefato ≠ rigor)**: balanceado — ADRs concisos, LIMITATIONS.md com itens verificáveis.
-

Veredito Final

APROVADO COM RESSALVAS para apresentação em entrevista.

O projeto exhibe maturidade acima do esperado para um MVP de 2 dias: decisões arquiteturais defensáveis, fail-closed na maioria dos caminhos, observabilidade estruturada, e — sobretudo — honestidade documentada sobre as limitações (diferencial sênior).

Antes da apresentação, recomenda-se corrigir os 3 críticos (C1, C2, C3) — são fixes de ~30-60 minutos cada e fecham buracos reais de fail-closed que um entrevistador rigoroso vai cutucar. Os médios e leves são munição para a conversa de "roadmap" e mostram visão sem comprometer o escopo MVP.

Forças narrativas para a entrevista: 1. Multi-provider via Protocols (ponte Bedrock crível, sem código morto) 2. Fail-closed em cascata (validator → node → API) 3. Red-teaming com fontes externas (anti closed-loop §1) 4. `LIMITATIONS.md` declarando o que falha (anti §7) 5. Pivots documentados em ADRs (mostra processo decisório, não engenheiro improvisador)

Risco principal a antecipar: se o entrevistador pedir "o que acontece se o Detoxify ou DeBERTa falha?", a resposta atual seria "request crasha" — daí a prioridade dos críticos C1/C2.